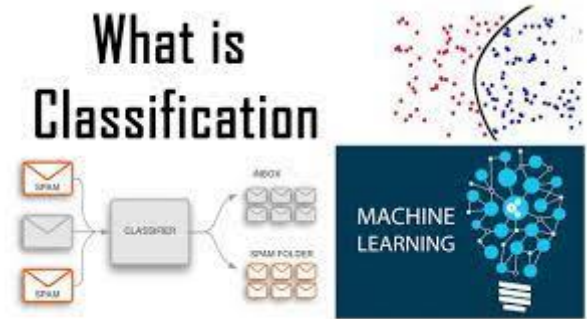


Unit-II

CLASSIFICATION



Classification Algorithm:

The Classification algorithm is a Supervised Learning technique that is used to identify the category of new observations on the basis of training data. In Classification, a program learns from the given dataset or observations and then classifies new observation into a number of classes or groups. Such as, Yes or No, 0 or 1, Spam or Not Spam, cat or dog, etc. Classes can be called as targets/labels or categories.

Classification is the task of “classifying things” into sub-categories. But, by a machine! If that doesn’t sound like much, imagine your computer being able to differentiate between you and a stranger. Between a potato and a tomato. Between an A grade and an F. Now, it sounds interesting now. In Machine Learning and Statistics, Classification is the problem of identifying to which of a set of categories (subpopulations), a new observation belongs, on the basis of a training set of data containing observations and whose categories membership is known.

Types of Classification:

Classification is of two types:

1. **Binary Classification:** When we have to categorize given data into 2 distinct classes.
Example – On the basis of given health conditions of a person, we have to determine whether the person has a certain disease or not.

2. **Multiclass Classification:** The number of classes is more than 2. For Example – On the basis of data about different species of flowers, we have to determine which specie our observation belongs.

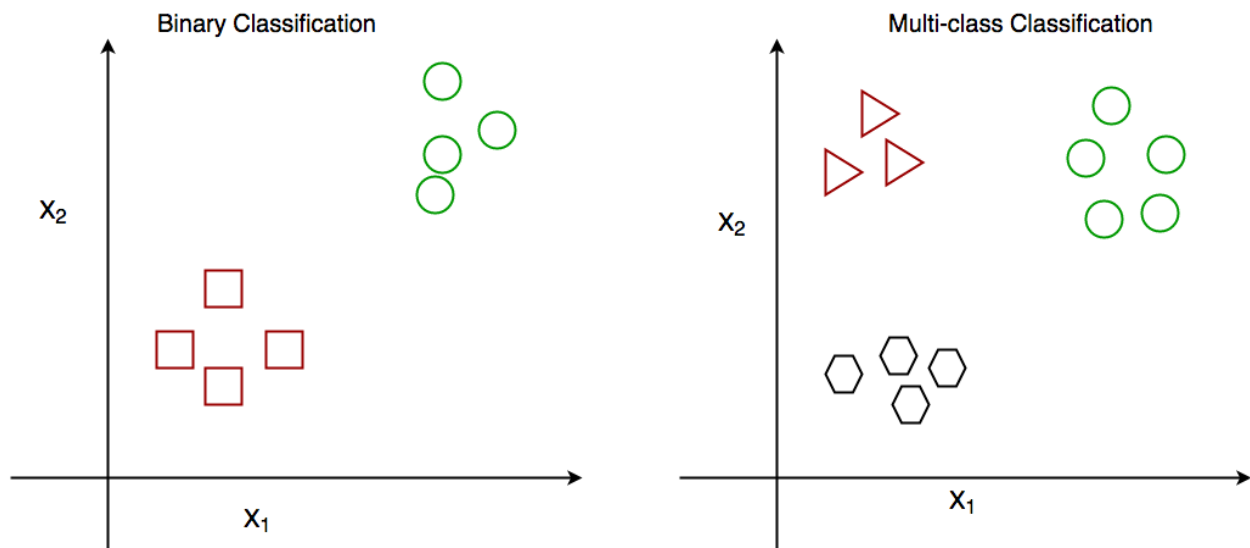


Fig: Binary and Multiclass Classification. Here x_1 and x_2 are the variables upon which the class is predicted.

How does classification works?

Suppose we have to predict whether a given patient has a certain disease or not, on the basis of 3 variables, called features.

This means there are two possible outcomes:

1. The patient has the said disease. Basically, a result labeled “Yes” or “True”.
2. The patient is disease-free. A result labeled “No” or “False”.

This is a binary classification problem.

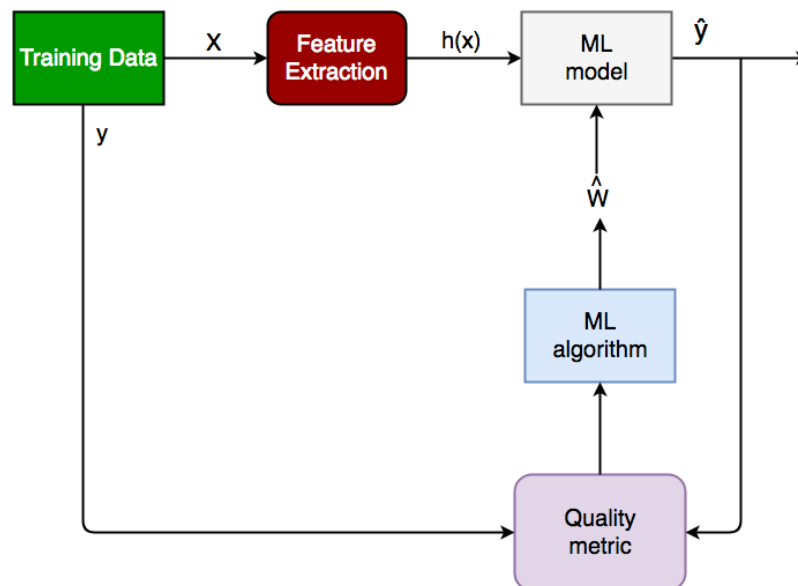
We have a set of observations called the training data set, which comprises sample data with actual classification results. We train a model, called Classifier on this data set,

and use that model to predict whether a certain patient will have the disease or not.

The outcome, thus now depends upon :

1. How well these features are able to “map” to the outcome.
2. The quality of our data set. By quality, I refer to statistical and Mathematical qualities.
3. How well our Classifier generalizes this relationship between the features and the outcome.
4. The values of the x_1 and x_2 .

Following is the generalized block diagram of the classification task.



Generalized Classification Block Diagram.

1. X : pre-classified data, in the form of an $N \times M$ matrix. N is the no. of observations and M is the number of features
2. y : An N -d vector corresponding to predicted classes for each of the N observations.
3. Feature Extraction: Extracting valuable information from input X using a series of transforms.
4. ML Model: The “Classifier” we’ll train.
5. y' : Labels predicted by the Classifier.
6. Quality Metric: Metric used for measuring the performance of the model.
7. ML Algorithm: The algorithm that is used to update weights w' , which updates the model and “learns” iteratively.

Types of Classifiers (algorithms): There are various types of classifiers

- Linear Classifiers: Logistic Regression
- Tree-Based Classifiers: Decision Tree Classifier
- Support Vector Machines
- Artificial Neural Networks
- Bayesian Regression
- Gaussian Naive Bayes Classifiers
- Stochastic Gradient Descent (SGD) Classifier
- Ensemble Methods: Random Forests, AdaBoost, Bagging Classifier, Voting Classifier, ExtraTrees Classifier

Practical Applications of Classification

1. Google's self-driving car uses deep learning-enabled classification techniques which enables it to detect and classify obstacles.
2. Spam E-mail filtering is one of the most widespread and well-recognized uses of Classification techniques.
3. Detecting Health Problems, Facial Recognition, Speech Recognition, Object Detection, and Sentiment Analysis all use Classification at their core.

Confusion Matrix:

The confusion matrix is a matrix used to determine the performance of the classification models for a given set of test data. It can only be determined if the true values for test data are known. The matrix itself can be easily understood, but the related terminologies may be confusing. Since it shows the errors in the model performance in the form of a matrix, hence also known as an **error matrix**. Some features of Confusion matrix are given below:

- For the 2 prediction classes of classifiers, the matrix is of 2*2 table, for 3 classes, it is 3*3 table, and so on.
- The matrix is divided into two dimensions, that are **predicted values** and **actual values** along with the total number of predictions.

- Predicted values are those values, which are predicted by the model, and actual values are the true values for the given observations.
- It looks like the below table:

n = total predictions	Actual: No	Actual: Yes
Predicted: No	True Negative	False Positive
Predicted: Yes	False Negative	True Positive

The above table has the following cases:

- **True Negative:** Model has given prediction No, and the real or actual value was also No.
- **True Positive:** The model has predicted yes, and the actual value was also true.
- **False Negative:** The model has predicted no, but the actual value was Yes, it is also called as **Type-II error**.
- **False Positive:** The model has predicted Yes, but the actual value was No. It is also called a **Type-I error**.

Need for Confusion Matrix in Machine learning

- It evaluates the performance of the classification models, when they make predictions on test data, and tells how good our classification model is.
- It not only tells the error made by the classifiers but also the type of errors such as it is either type-I or type-II error.
- With the help of the confusion matrix, we can calculate the different parameters for the model, such as accuracy, precision, etc.

Example: We can understand the confusion matrix using an example.

Suppose we are trying to create a model that can predict the result for the disease that is either a person has that disease or not. So, the confusion matrix for this is given as:

n = 100	Actual: No	Actual: Yes	
Predicted: No	TN: 65	FP: 3	68
Predicted: Yes	FN: 8	TP: 24	32
	73	27	

From the above example, we can conclude that:

- The table is given for the two-class classifier, which has two predictions "Yes" and "NO." Here, Yes defines that patient has the disease, and No defines that patient does not has that disease.
- The classifier has made a total of **100 predictions**. Out of 100 predictions, **89 are true predictions**, and **11 are incorrect predictions**.
- The model has given prediction "yes" for 32 times, and "No" for 68 times. Whereas the actual "Yes" was 27, and actual "No" was 73 times.

Calculations using Confusion Matrix:

We can perform various calculations for the model, such as the model's accuracy, using this matrix. These calculations are given below:

- **Classification Accuracy:** It is one of the important parameters to determine the accuracy of the classification problems. It defines how often the model predicts the correct output. It can be calculated as the ratio of the number of correct predictions made by the classifier to all number of predictions made by the classifiers. The formula is given below:

$$\text{Accuracy} = \frac{TP+TN}{TP+FP+FN+TN}$$

- **Misclassification rate:** It is also termed as Error rate, and it defines how often the model gives the wrong predictions. The value of error rate can be calculated as the number of incorrect predictions to all number of the predictions made by the classifier. The formula is given below:

$$\text{Error rate} = \frac{FP + FN}{TP + FP + FN + TN}$$

- **Precision:** It can be defined as the number of correct outputs provided by the model or out of all positive classes that have predicted correctly by the model, how many of them were actually true. It can be calculated using the below formula:

$$\text{Precision} = \frac{TP}{TP + FP}$$

- **Recall:** It is defined as the out of total positive classes, how our model predicted correctly. The recall must be as high as possible.

$$\text{Recall} = \frac{TP}{TP + FN}$$

- **F-measure:** If two models have low precision and high recall or vice versa, it is difficult to compare these models. So, for this purpose, we can use F-score. This score helps us to evaluate the recall and precision at the same time. The F-score is maximum if the recall is equal to the precision. It can be calculated using the below formula:

$$\text{F-measure} = \frac{2 * \text{Recall} * \text{Precision}}{\text{Recall} + \text{Precision}}$$

Multi Class Classification Models:

Many machine learning algorithms can be used to train a multiclass classifier but not all as standard algorithms such as logistic regression, support vector machines (SVM) are designed only for binary classification tasks. However, one can use many strategies to leverage these traditional algorithms in multiclass classification.

1. Binary Transformation

The entire task is broken into multiple binary classification problems using strategies like one-vs-rest and one-vs-one to use them in multiclass classification.

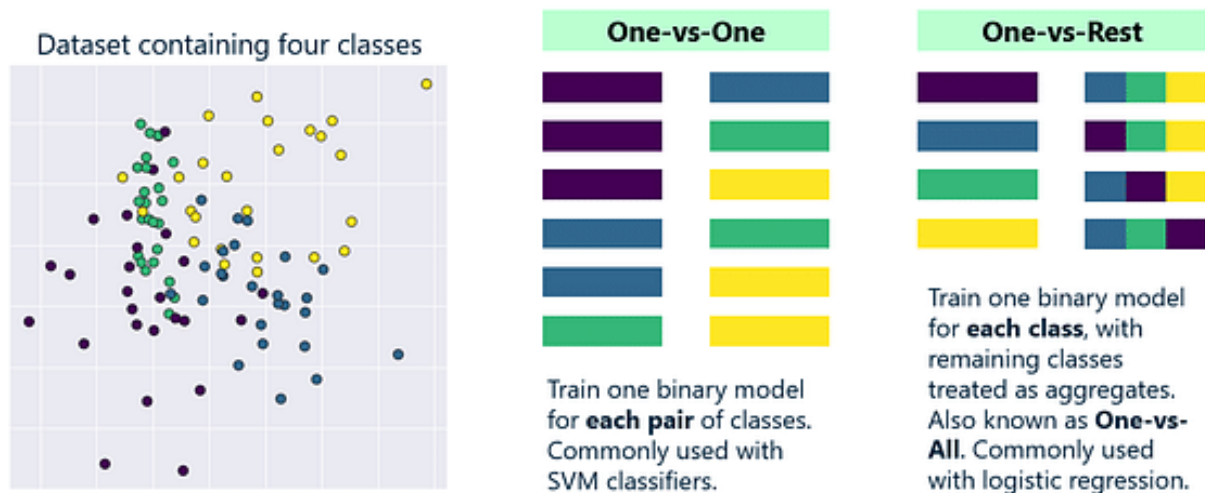


Fig. wintellect.com

One-vs-Rest

As the name suggests, we train a binary classifier for every class where the class distribution is the target class versus the rest of the classes.

For example, if you are working on a problem of predicting whether the given fruit is an apple, mango, or banana, you will train three binary classifiers.

1. Classifier A: [apple] v/s [mango, banana]
2. Classifier B: [mango] v/s [apple, banana]
3. Classifier C: [banana] v/s [apple, mango]

Voting is used to predict the final class, i.e., the binary classifier predicting the target class with the highest confidence is given as the final output and is commonly used with Logistic Regression.

One-vs-One

In the approach, a binary classifier is trained for every pair of classes, i.e., one class versus every other class. The number of binary classifiers to be trained can be calculated with the help of this simple formula:

$(N * (N-1))/2$ where N = total number of classes

For example, taking the model above, the total classifiers to be trained are three, which are as follows:

1. Classifier A: apple v/s mango
2. Classifier B: apple v/s banana
3. Classifier C: mango v/s banana

Each binary classifier predicts one class, and the class predicted by the majority of the classifiers is given as the final class prediction. This strategy works the best for SVM or other kernel-based algorithms.

Computing Precision and Recall for the Multi-Class Problem

While it is fairly straightforward to compute precision and recall for a binary classification problem, it can be quite confusing as to how to compute these values for a multi-class classification problem. Now let's look at how to compute precision and recall for a multi-class problem.

- First, let us assume that we have a 3-class multi classification problem, with labels A, B and C.
- The first thing to do is to generate a confusion matrix as below. Many existing machine learning packages already generate the confusion matrix for you, but if you don't have that luxury, it is actually very easy to implement it yourself by keeping counters for the true positives, false positives and total number of instances for each label.

	GoldLabel_A	GoldLabel_B	GoldLabel_C	
Predicted_A	30	20	10	TotalPredicted_A=60
Predicted_B	50	60	10	TotalPredicted_B=120
Predicted_C	20	20	80	TotalPredicted_C=120
	TotalGoldLabel_A=100	TotalGoldLabel_B=100	TotalGoldLabel_C=100	

This is an example confusion matrix for 3 labels: A,B and C

- Once you have the confusion matrix, you have all the values you need to compute precision and recall for each class. Note that the values in the diagonal would always be the true positives (TP).

Now, let us compute **recall** for Label A:

$$\begin{aligned}
 &= TP_A / (TP_A + FN_A) \\
 &= TP_A / (\text{Total Gold for A}) \\
 &= TP_A / \text{TotalGoldLabel_A} \\
 &= 30 / 100 \\
 &= 0.3
 \end{aligned}$$

Now, let us compute **precision** for Label A:

$$\begin{aligned}
 &= TP_A / (TP_A + FP_A) \\
 &= TP_A / (\text{Total predicted as A}) \\
 &= TP_A / \text{TotalPredicted_A} \\
 &= 30 / 60 \\
 &= 0.5
 \end{aligned}$$

So **precision=0.5** and **recall=0.3** for label A. Which means that for precision, out of the times label A was predicted, 50% of the time the system was in fact correct. And for recall, it means that out of all the times label A should have been predicted only 30% of the labels were correctly predicted.

Now, let us compute **recall** for Label B:

```
= TP_B / (TP_B + FN_B)
= TP_B / (Total Gold for B)
= TP_B / TotalGoldLabel_B
= 60 / 100
= 0.6
```

Now, let us compute **precision** for Label B:

```
= TP_B / (TP_B + FP_B)
= TP_B / (Total predicted as B)
= TP_B / TotalPredicted_B
= 60 / 120
= 0.5
```

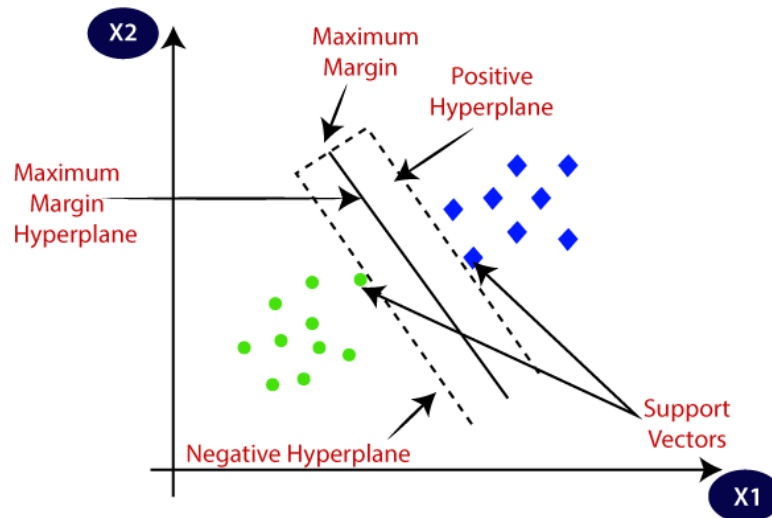
So **precision=0.5** and **recall=0.6** for label B. So you just have to repeat this for each label in your multi-class classification problem.

Support Vector Machine Algorithm:

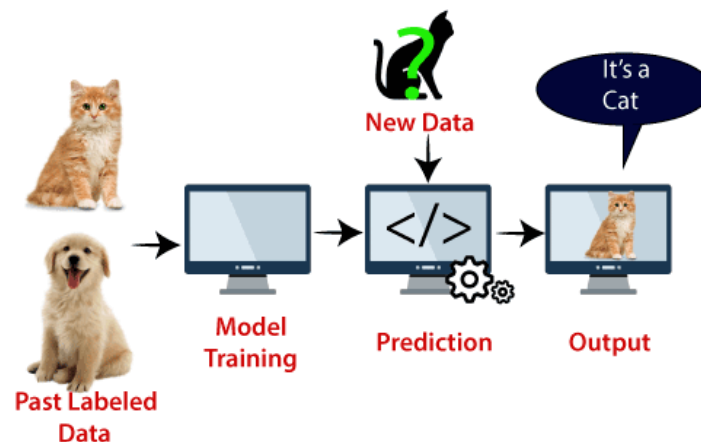
Support Vector Machine or SVM is one of the most popular Supervised Learning algorithms, which is used for Classification as well as Regression problems. However, primarily, it is used for Classification problems in Machine Learning.

The goal of the SVM algorithm is to create the best line or decision boundary that can segregate n-dimensional space into classes so that we can easily put the new data point in the correct category in the future. This best decision boundary is called a hyperplane.

SVM chooses the extreme points/vectors that help in creating the hyperplane. These extreme cases are called as support vectors, and hence algorithm is termed as Support Vector Machine. Consider the below diagram in which there are two different categories that are classified using a decision boundary or hyperplane:



Example: SVM can be understood with the example that we have used in the KNN classifier. Suppose we see a strange cat that also has some features of dogs, so if we want a model that can accurately identify whether it is a cat or dog, so such a model can be created by using the SVM algorithm. We will first train our model with lots of images of cats and dogs so that it can learn about different features of cats and dogs, and then we test it with this strange creature. So as support vector creates a decision boundary between these two data (cat and dog) and choose extreme cases (support vectors), it will see the extreme case of cat and dog. On the basis of the support vectors, it will classify it as a cat. Consider the below diagram:



SVM algorithm can be used for **Face detection, image classification, text categorization**, etc.

Types of SVM

SVM can be of two types:

- **Linear SVM:** Linear SVM is used for linearly separable data, which means if a dataset can be classified into two classes by using a single straight line, then such data is termed as linearly separable data, and classifier is used called as Linear SVM classifier.
- **Non-linear SVM:** Non-Linear SVM is used for non-linearly separated data, which means if a dataset cannot be classified by using a straight line, then such data is termed as non-linear data and classifier used is called as Non-linear SVM classifier.

Hyperplane and Support Vectors in the SVM algorithm:

Hyperplane: There can be multiple lines/decision boundaries to segregate the classes in n-dimensional space, but we need to find out the best decision boundary that helps to classify the data points. This best boundary is known as the hyperplane of SVM.

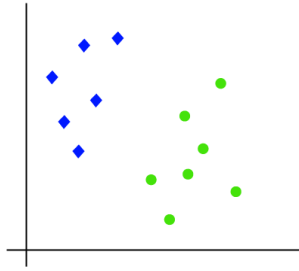
The dimensions of the hyperplane depend on the features present in the dataset, which means if there are 2 features (as shown in image), then hyperplane will be a straight line. And if there are 3 features, then hyperplane will be a 2-dimension plane. We always create a hyperplane that has a maximum margin, which means the maximum distance between the data points.

Support Vectors:

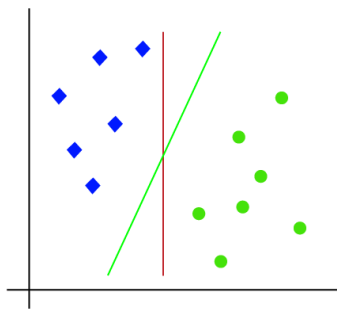
The data points or vectors that are the closest to the hyperplane and which affect the position of the hyperplane are termed as Support Vector. Since these vectors support the hyperplane, hence called a Support vector.

How does SVM works?

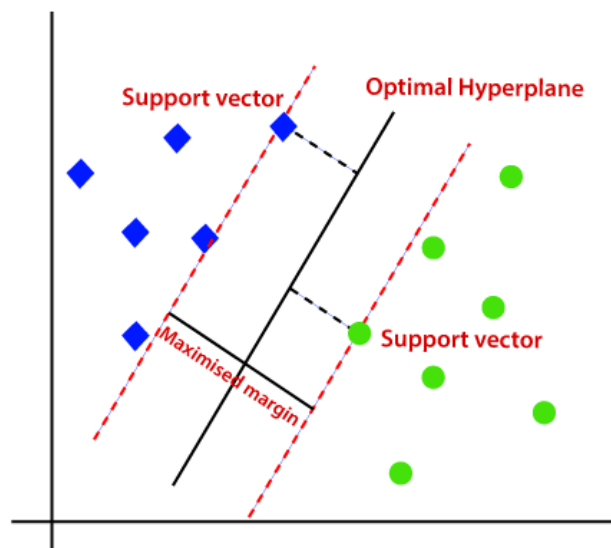
Linear SVM: The working of the SVM algorithm can be understood by using an example. Suppose we have a dataset that has two tags (green and blue), and the dataset has two features x_1 and x_2 . We want a classifier that can classify the pair(x_1 , x_2) of coordinates in either green or blue. Consider the below image:



So as it is 2-d space so by just using a straight line, we can easily separate these two classes. But there can be multiple lines that can separate these classes. Consider the below image:

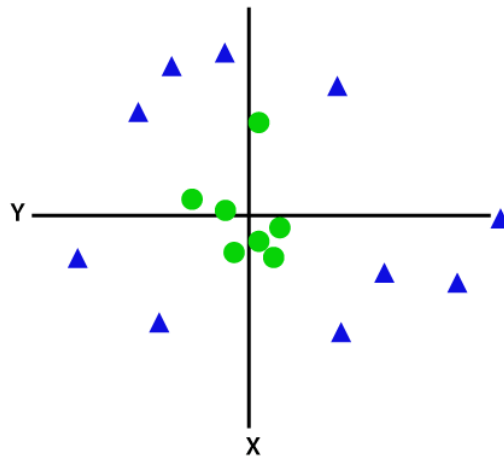


Hence, the SVM algorithm helps to find the best line or decision boundary; this best boundary or region is called as a **hyperplane**. SVM algorithm finds the closest point of the lines from both the classes. These points are called support vectors. The distance between the vectors and the hyperplane is called as **margin**. And the goal of SVM is to maximize this margin. The **hyperplane** with maximum margin is called the **optimal hyperplane**.



Non-Linear SVM:

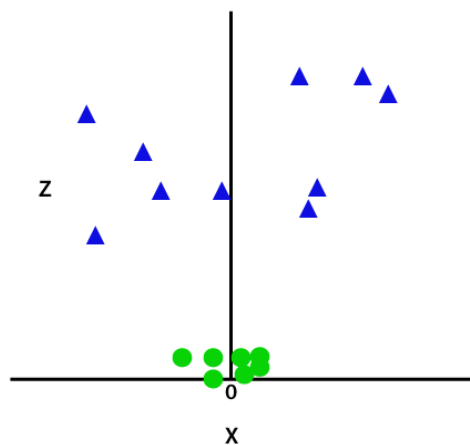
If data is linearly arranged, then we can separate it by using a straight line, but for non-linear data, we cannot draw a single straight line. Consider the below image:



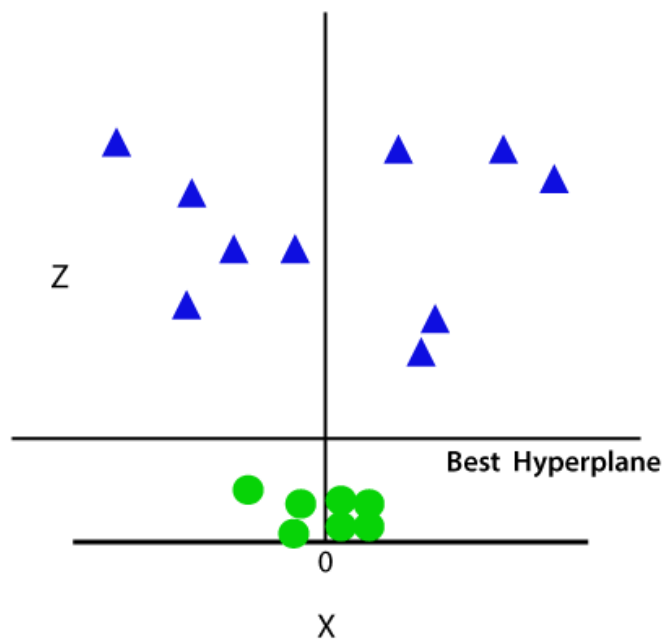
So to separate these data points, we need to add one more dimension. For linear data, we have used two dimensions x and y, so for non-linear data, we will add a third dimension z. It can be calculated as:

$$z = x^2 + y^2$$

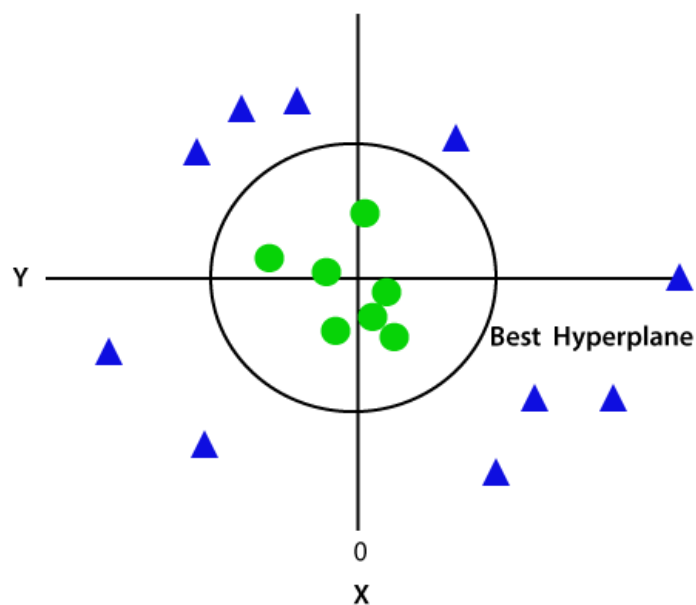
By adding the third dimension, the sample space will become as below image:



So now, SVM will divide the datasets into classes in the following way. Consider the below image:



Since we are in 3-d Space, hence it is looking like a plane parallel to the x-axis. If we convert it in 2d space with $z=1$, then it will become as:



Hence we get a circumference of radius 1 in case of non-linear data.

Kernel Functions in Support Vector Machine (SVM)

Kernel Function is a method used to take data as input and transform it into the required form of processing data. “Kernel” is used due to a set of mathematical functions used in Support Vector Machine providing the window to manipulate the data. So, Kernel Function generally transforms the training set of data so that a non-linear decision surface is able to transform to a linear equation in a higher number of dimension spaces.

Standard Kernel Function Equation:

$$K(\bar{x}) = 1, \text{ if } \|\bar{x}\| \leq 1$$

$$K(\bar{x}) = 0, \text{ Otherwise}$$

Gaussian Kernel: It is used to perform transformation when there is no prior knowledge about data.

$$K(x, y) = e^{-\left(\frac{\|x-y\|^2}{2\sigma^2}\right)}$$

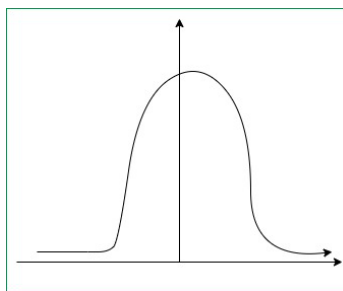
- **Gaussian Kernel Radial Basis Function (RBF):** Same as above kernel function, adding radial basis method to improve the transformation.

$$K(x, y) = e^{-\gamma\|x - y\|^2}$$

$$K(x, x1) + K(x, x2) \text{ (Simplified - Formula)}$$

$$K(x, x1) + K(x, x2) > 0 \text{ (Green)}$$

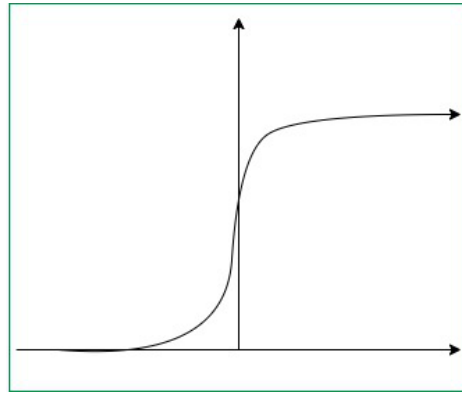
$$K(x, x1) + K(x, x2) = 0 \text{ (Red)}$$



Gaussian Kernel Graph

- **Sigmoid Kernel:** this function is equivalent to a two-layer, perceptron model of the neural network, which is used as an activation function for artificial neurons.

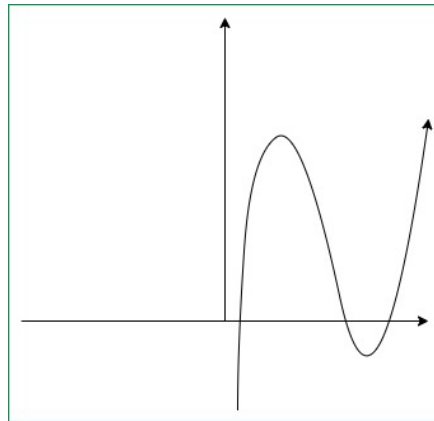
$$K(x, y) = \tanh(\gamma x^T y + r)$$



Sigmoid Kernel Graph

- **Polynomial Kernel:** It represents the similarity of vectors in the training set of data in a feature space over polynomials of the original variables used in the kernel.

$$K(x, y) = \tanh(\gamma x^T y + r)^d, \gamma > 0$$



Polynomial Kernel Graph